JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class JapaneseChronology

java.lang.Object
  java.time.chrono.AbstractChronology
    java.time.chrono.JapaneseChronology

All Implemented Interfaces:

`Serializable`, `Comparable<Chronology>`, `Chronology`

```
public final class JapaneseChronology
extends AbstractChronology
implements Serializable
```

The Japanese Imperial calendar system.

This chronology defines the rules of the Japanese Imperial calendar system. This calendar system is primarily used in Japan. The Japanese Imperial calendar system is the same as the ISO calendar system apart from the era-based year numbering.

Japan introduced the Gregorian calendar starting with Meiji 6. Only Meiji and later eras are supported; dates before Meiji 6, January 1 are not supported.

The supported `ChronoField` instances are:

- `DAY_OF_WEEK`
- `DAY_OF_MONTH`
- `DAY_OF_YEAR`
- `EPOCH_DAY`
- `MONTH_OF_YEAR`
- `PROLEPTIC_MONTH`
- `YEAR_OF_ERA`
- `YEAR`
- `ERA`

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description

static final JapaneseChronology	**INSTANCE**	Singleton instance for Japanese chronology.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
JapaneseDate	**date**(int prolepticYear, int month, int dayOfMonth)	Obtains a local date in Japanese calendar system from the proleptic-year, month-of-year and day-of-month fields.
JapaneseDate	**date**(Era era, int yearOfEra, int month, int dayOfMonth)	Obtains a local date in Japanese calendar system from the era, year-of-era, month-of-year and day-of-month fields.
JapaneseDate	**date**(TemporalAccessor tempor	Obtains a local date in this chronology from another temporal object.
JapaneseDate	**dateEpochDay**(long epochDay)	Obtains a local date in the Japanese calendar system from the epoch-day.
JapaneseDate	**dateNow**()	Obtains the current local date in this chronology from the system clock in the default time-zone.
JapaneseDate	**dateNow**(Clock clock)	Obtains the current local date in this chronology from the specified clock.
JapaneseDate	**dateNow**(ZoneId zone)	Obtains the current local date in this chronology from the system clock in the specified time-zone.

JapaneseDate	**dateYearDay** (int prolepticYear, int dayOfYear)	Obtains a local date in Japanese calendar system from the proleptic-year and day-of-year fields.
JapaneseDate	**dateYearDay**(Era era, int yearOfEra, int dayOfYear)	Obtains a local date in Japanese calendar system from the era, year-of-era and day-of-year fields.
JapaneseEra	**eraOf**(int eraValue)	Returns the calendar system era object from the given numeric value.
List<Era>	**eras**()	Gets the list of eras for the chronology.
String	**getCalendarType**()	Gets the calendar type of the underlying calendar system - 'japanese'.
String	**getId**()	Gets the ID of the chronology - 'Japanese'.
boolean	**isIsoBased**()	JapaneseChronology is an ISO based chronology, which supports fields in IsoFields, such as DAY_OF_QUARTER and QUARTER_OF_YEAR.
boolean	**isLeapYear** (long prolepticYear)	Checks if the specified year is a leap year.
ChronoLocalDateTime <JapaneseDate>	**localDateTime** (TemporalAccessor tempor	Obtains a local date-time in this chronology from another temporal object.
int	**prolepticYear**(Era era, int yearOfEra)	Calculates the proleptic-year given the era and year-of-era.
ValueRange	**range** (ChronoField field)	

		Gets the range of valid values for the specified field.
JapaneseDate	**resolveDate**(Map<TemporalField, Long> fieldValues, ResolverStyle resolverSt	Resolves parsed ChronoField values into a date during parsing.
ChronoZonedDateTime<JapaneseDate>	**zonedDateTime**(Instant instant, ZoneId zone)	Obtains a ChronoZonedDateTime in this chronology from an Instant.
ChronoZonedDateTime<JapaneseDate>	**zonedDateTime**(TemporalAccessor tempor	Obtains a ChronoZonedDateTime in this chronology from another temporal object.

Methods declared in class AbstractChronology

compareTo, equals, hashCode, toString

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods declared in interface Chronology

epochSecond, epochSecond, getDisplayName, period

Field Details

INSTANCE

```
public static final JapaneseChronology INSTANCE
```

Singleton instance for Japanese chronology.

Method Details

getId

```
public String getId()
```

Gets the ID of the chronology - 'Japanese'.

The ID uniquely identifies the `Chronology`. It can be used to lookup the `Chronology` using `Chronology.of(String)`.

Specified by:

`getId` in interface `Chronology`

Returns:

the chronology ID - 'Japanese'

See Also:

`getCalendarType()`

getCalendarType

```
public String getCalendarType()
```

Gets the calendar type of the underlying calendar system - 'japanese'.

The calendar type is an identifier defined by the *Unicode Locale Data Markup Language (LDML)* specification. It can be used to lookup the `Chronology` using `Chronology.of(String)`. It can also be used as part of a locale, accessible via `Locale.getUnicodeLocaleType(String)` with the key 'ca'.

Specified by:

`getCalendarType` in interface `Chronology`

Returns:

the calendar system type - 'japanese'

See Also:

`getId()`

date

```
public JapaneseDate date(Era era,
                         int yearOfEra,
                         int month,
                         int dayOfMonth)
```

Obtains a local date in Japanese calendar system from the era, year-of-era, month-of-year and day-of-month fields.

The Japanese month and day-of-month are the same as those in the ISO calendar system. They are not reset when the era changes. For example:

```
6th Jan Showa 64 = ISO 1989-01-06
7th Jan Showa 64 = ISO 1989-01-07
```

```
8th Jan Heisei 1 = ISO 1989-01-08
9th Jan Heisei 1 = ISO 1989-01-09
```

Specified by:

`date` in interface `Chronology`

Parameters:

`era` - the Japanese era, not null

`yearOfEra` - the year-of-era

`month` - the month-of-year

`dayOfMonth` - the day-of-month

Returns:

the Japanese local date, not null

Throws:

`DateTimeException` - if unable to create the date

`ClassCastException` - if the `era` is not a `JapaneseEra`

date

```
public JapaneseDate date(int prolepticYear,
                         int month,
                         int dayOfMonth)
```

Obtains a local date in Japanese calendar system from the proleptic-year, month-of-year and day-of-month fields.

The Japanese proleptic year, month and day-of-month are the same as those in the ISO calendar system. They are not reset when the era changes.

Specified by:

`date` in interface `Chronology`

Parameters:

`prolepticYear` - the proleptic-year

`month` - the month-of-year

`dayOfMonth` - the day-of-month

Returns:

the Japanese local date, not null

Throws:

`DateTimeException` - if unable to create the date

dateYearDay

```
public JapaneseDate dateYearDay(Era era,
                                int yearOfEra,
                                int dayOfYear)
```

Obtains a local date in Japanese calendar system from the era, year-of-era and day-of-year fields.

The day-of-year in this factory is expressed relative to the start of the year-of-era. This definition changes the normal meaning of day-of-year only in those years where the year-of-era is reset to one due to a change in the era. For example:

```
6th Jan Showa 64 = day-of-year 6
7th Jan Showa 64 = day-of-year 7
8th Jan Heisei 1 = day-of-year 1
9th Jan Heisei 1 = day-of-year 2
```

Specified by:

`dateYearDay` in interface `Chronology`

Parameters:

`era` - the Japanese era, not null

`yearOfEra` - the year-of-era

`dayOfYear` - the day-of-year

Returns:

the Japanese local date, not null

Throws:

`DateTimeException` - if unable to create the date

`ClassCastException` - if the `era` is not a `JapaneseEra`

dateYearDay

```
public JapaneseDate dateYearDay(int prolepticYear,
                                int dayOfYear)
```

Obtains a local date in Japanese calendar system from the proleptic-year and day-of-year fields.

The day-of-year in this factory is expressed relative to the start of the proleptic year. The Japanese proleptic year and day-of-year are the same as those in the ISO calendar system. They are not reset when the era changes.

Specified by:

dateYearDay in interface Chronology

Parameters:

prolepticYear - the proleptic-year

dayOfYear - the day-of-year

Returns:

the Japanese local date, not null

Throws:

DateTimeException - if unable to create the date

dateEpochDay

```
public JapaneseDate dateEpochDay(long epochDay)
```

Obtains a local date in the Japanese calendar system from the epoch-day.

Specified by:

dateEpochDay in interface Chronology

Parameters:

epochDay - the epoch day

Returns:

the Japanese local date, not null

Throws:

DateTimeException - if unable to create the date

dateNow

```
public JapaneseDate dateNow()
```

Description copied from interface: Chronology

Obtains the current local date in this chronology from the system clock in the default time-zone.

This will query the system clock in the default time-zone to obtain the current date.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Specified by:

dateNow in interface Chronology

Returns:

the current local date using the system clock and default time-zone, not null

dateNow

```
public JapaneseDate dateNow(ZoneId zone)
```

Description copied from interface: Chronology

Obtains the current local date in this chronology from the system clock in the specified time-zone.

This will query the `system clock` to obtain the current date. Specifying the time-zone avoids dependence on the default time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Specified by:

`dateNow` in interface `Chronology`

Parameters:

`zone` - the zone ID to use, not null

Returns:

the current local date using the system clock, not null

dateNow

```
public JapaneseDate dateNow(Clock clock)
```

Description copied from interface: Chronology

Obtains the current local date in this chronology from the specified clock.

This will query the specified clock to obtain the current date - today. Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using `dependency injection`.

Specified by:

`dateNow` in interface `Chronology`

Parameters:

`clock` - the clock to use, not null

Returns:

the current local date, not null

date

```
public JapaneseDate date(TemporalAccessor temporal)
```

Description copied from interface: Chronology

Obtains a local date in this chronology from another temporal object.

This obtains a date in this chronology based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ChronoLocalDate`.

The conversion typically uses the `EPOCH_DAY` field, which is standardized across calendar systems.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `aChronology::date`.

Specified by:

`date` in interface `Chronology`

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the local date in this chronology, not null

See Also:

`ChronoLocalDate.from(TemporalAccessor)`

localDateTime

```
public ChronoLocalDateTime<JapaneseDate> localDateTime
(TemporalAccessor temporal)
```

Description copied from interface: `Chronology`

Obtains a local date-time in this chronology from another temporal object.

This obtains a date-time in this chronology based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ChronoLocalDateTime`.

The conversion extracts and combines the `ChronoLocalDate` and the `LocalTime` from the temporal object. Implementations are permitted to perform optimizations such as accessing those fields that are equivalent to the relevant objects. The result uses this chronology.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `aChronology::localDateTime`.

Specified by:

`localDateTime` in interface `Chronology`

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the local date-time in this chronology, not null

See Also:

`ChronoLocalDateTime.from(TemporalAccessor)`

zonedDateTime

```
public ChronoZonedDateTime<JapaneseDate> zonedDateTime
(TemporalAccessor temporal)
```

Description copied from interface: `Chronology`

Obtains a `ChronoZonedDateTime` in this chronology from another temporal object.

This obtains a zoned date-time in this chronology based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ChronoZonedDateTime`.

The conversion will first obtain a `ZoneId` from the temporal object, falling back to a `ZoneOffset` if necessary. It will then try to obtain an `Instant`, falling back to a `ChronoLocalDateTime` if necessary. The result will be either the combination of `ZoneId` or `ZoneOffset` with `Instant` or `ChronoLocalDateTime`. Implementations are permitted to perform optimizations such as accessing those fields that are equivalent to the relevant objects. The result uses this chronology.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `aChronology::zonedDateTime`.

Specified by:

`zonedDateTime` in interface `Chronology`

Parameters:

`temporal` - the temporal object to convert, not null

Returns:

the zoned date-time in this chronology, not null

See Also:

`ChronoZonedDateTime.from(TemporalAccessor)`

zonedDateTime

```
public ChronoZonedDateTime<JapaneseDate> zonedDateTime
(Instant instant,
 ZoneId zone)
```

Description copied from interface: `Chronology`

Obtains a `ChronoZonedDateTime` in this chronology from an `Instant`.

This obtains a zoned date-time with the same instant as that specified.

Specified by:

`zonedDateTime` in interface `Chronology`

Parameters:

`instant` - the instant to create the date-time from, not null

`zone` - the time-zone, not null

Returns:

the zoned date-time, not null

isLeapYear

```
public boolean isLeapYear(long prolepticYear)
```

Checks if the specified year is a leap year.

Japanese calendar leap years occur exactly in line with ISO leap years. This method does not validate the year passed in, and only has a well-defined result for years in the supported range.

Specified by:

`isLeapYear` in interface `Chronology`

Parameters:

`prolepticYear` - the proleptic-year to check, not validated for range

Returns:

true if the year is a leap year

prolepticYear

```
public int prolepticYear(Era era,
                         int yearOfEra)
```

Description copied from interface: `Chronology`

Calculates the proleptic-year given the era and year-of-era.

This combines the era and year-of-era into the single proleptic-year field.

If the chronology makes active use of eras, such as `JapaneseChronology` then the year-of-era will be validated against the era. For other chronologies, validation is optional.

Specified by:

prolepticYear in interface Chronology

Parameters:

era - the era of the correct type for the chronology, not null

yearOfEra - the chronology year-of-era

Returns:

the proleptic-year

eraOf

```
public JapaneseEra eraOf(int eraValue)
```

Returns the calendar system era object from the given numeric value. The numeric values supported by this method are the same as the numeric values supported by JapaneseEra.of(int).

Specified by:

eraOf in interface Chronology

Parameters:

eraValue - the era value

Returns:

the Japanese Era for the given numeric era value

Throws:

DateTimeException - if eraValue is invalid

eras

```
public List<Era> eras()
```

Description copied from interface: Chronology

Gets the list of eras for the chronology.

Most calendar systems have an era, within which the year has meaning. If the calendar system does not support the concept of eras, an empty list must be returned.

Specified by:

eras in interface Chronology

Returns:

the list of eras for the chronology, may be immutable, not null

range

```
public ValueRange range(ChronoField field)
```

Description copied from interface: `Chronology`

Gets the range of valid values for the specified field.

All fields can be expressed as a `long` integer. This method returns an object that describes the valid range for that value.

Note that the result only describes the minimum and maximum valid values and it is important not to read too much into them. For example, there could be values within the range that are invalid for the field.

This method will return a result whether or not the chronology supports the field.

Specified by:

`range` in interface `Chronology`

Parameters:

`field` - the field to get the range for, not null

Returns:

the range of valid values for the field, not null

resolveDate

```
public JapaneseDate resolveDate(Map<TemporalField, Long> fieldValues,
                                ResolverStyle resolverStyle)
```

Description copied from class: `AbstractChronology`

Resolves parsed `ChronoField` values into a date during parsing.

Most `TemporalField` implementations are resolved using the resolve method on the field. By contrast, the `ChronoField` class defines fields that only have meaning relative to the chronology. As such, `ChronoField` date fields are resolved here in the context of a specific chronology.

`ChronoField` instances are resolved by this method, which may be overridden in subclasses.

- `EPOCH_DAY` - If present, this is converted to a date and all other date fields are then cross-checked against the date.
- `PROLEPTIC_MONTH` - If present, then it is split into the `YEAR` and `MONTH_OF_YEAR`. If the mode is strict or smart then the field is validated.
- `YEAR_OF_ERA` and `ERA` - If both are present, then they are combined to form a `YEAR`. In lenient mode, the `YEAR_OF_ERA` range is not validated, in smart and strict mode it is. The `ERA` is validated for range in all three modes. If only the `YEAR_OF_ERA` is present, and the mode is smart or lenient, then the last available era is assumed. In strict mode, no era is

assumed and the YEAR_OF_ERA is left untouched. If only the ERA is present, then it is left untouched.

- YEAR, MONTH_OF_YEAR and DAY_OF_MONTH - If all three are present, then they are combined to form a date. In all three modes, the YEAR is validated. If the mode is smart or strict, then the month and day are validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first day of the first month in the requested year, then adding the difference in months, then the difference in days. If the mode is smart, and the day-of-month is greater than the maximum for the year-month, then the day-of-month is adjusted to the last day-of-month. If the mode is strict, then the three fields must form a valid date.
- YEAR and DAY_OF_YEAR - If both are present, then they are combined to form a date. In all three modes, the YEAR is validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first day of the requested year, then adding the difference in days. If the mode is smart or strict, then the two fields must form a valid date.
- YEAR, MONTH_OF_YEAR, ALIGNED_WEEK_OF_MONTH and ALIGNED_DAY_OF_WEEK_IN_MONTH - If all four are present, then they are combined to form a date. In all three modes, the YEAR is validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first day of the first month in the requested year, then adding the difference in months, then the difference in weeks, then in days. If the mode is smart or strict, then the all four fields are validated to their outer ranges. The date is then combined in a manner equivalent to creating a date on the first day of the requested year and month, then adding the amount in weeks and days to reach their values. If the mode is strict, the date is additionally validated to check that the day and week adjustment did not change the month.
- YEAR, MONTH_OF_YEAR, ALIGNED_WEEK_OF_MONTH and DAY_OF_WEEK - If all four are present, then they are combined to form a date. The approach is the same as described above for years, months and weeks in ALIGNED_DAY_OF_WEEK_IN_MONTH. The day-of-week is adjusted as the next or same matching day-of-week once the years, months and weeks have been handled.
- YEAR, ALIGNED_WEEK_OF_YEAR and ALIGNED_DAY_OF_WEEK_IN_YEAR - If all three are present, then they are combined to form a date. In all three modes, the YEAR is validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first day of the requested year, then adding the difference in weeks, then in days. If the mode is smart or strict, then the all three fields are validated to their outer ranges. The date is then combined in a manner equivalent to creating a date on the first day of the requested year, then adding the amount in weeks and days to reach their values. If the mode is strict, the

date is additionally validated to check that the day and week adjustment did not change the year.
- `YEAR`, `ALIGNED_WEEK_OF_YEAR` and `DAY_OF_WEEK` - If all three are present, then they are combined to form a date. The approach is the same as described above for years and weeks in `ALIGNED_DAY_OF_WEEK_IN_YEAR`. The day-of-week is adjusted as the next or same matching day-of-week once the years and weeks have been handled.

The default implementation is suitable for most calendar systems. If `ChronoField.YEAR_OF_ERA` is found without an `ChronoField.ERA` then the last era in `Chronology.eras()` is used. The implementation assumes a 7 day week, that the first day-of-month has the value 1, that first day-of-year has the value 1, and that the first of the month and year always exists.

Specified by:

`resolveDate` in interface `Chronology`

Overrides:

`resolveDate` in class `AbstractChronology`

Parameters:

`fieldValues` - the map of fields to values, which can be updated, not null

`resolverStyle` - the requested type of resolve, not null

Returns:

the resolved date, null if insufficient information to create a date

isIsoBased

```
public boolean isIsoBased()
```

`JapaneseChronology` is an ISO based chronology, which supports fields in `IsoFields`, such as `DAY_OF_QUARTER` and `QUARTER_OF_YEAR`.

Specified by:

`isIsoBased` in interface `Chronology`

Returns:

`true`

Since:

19

See Also:

`IsoFields`

---

contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.

All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.